

TaskFlow — Frontend Implementation Guide

Companion to SRS v1.0 | April 2026

Design Philosophy

TaskFlow is a productivity tool. The aesthetic should be **refined utility** — dark-first, data-dense without feeling cluttered, with subtle motion that communicates state changes rather than decorating them. Think Linear.app crossed with a terminal-inspired monospace accent. Every animation should have a functional reason: confirming an action, indicating loading, communicating hierarchy.

Tone: Calm, focused, professional. The UI should get out of the user's way.

1. Design System

1.1 Color Palette (CSS Variables)

Use these in your `src/styles/globals.css` or as a Tailwind theme extension:

CSS

```
:root {  
  /* Base */  
  --bg-base:      #0d0d0f;    /* Page background */  
  --bg-surface:    #17171a;    /* Cards, panels */  
  --bg-overlay:    #1f1f24;    /* Modals, dropdowns */  
  --bg-hover:      #26262c;    /* Hover states */  
  --bg-active:     #2e2e36;    /* Active / selected */  
  
  /* Borders */  
  --border-subtle: #2a2a32;    /* Dividers, table rows */  
  --border-default: #3a3a45;   /* Input outlines, card edges */  
  --border-strong: #52525e;    /* Focus rings */  
  
  /* Text */  
  --text-primary:  #f0f0f5;    /* Headings, body text */  
  --text-secondary: #9898a8;    /* Labels, metadata */  
  --text-muted:    #5e5e6e;    /* Placeholders, disabled */  
  
  /* Brand Accent */  
  --accent:        #6e6af0;    /* Primary buttons, links, focus */  
  --accent-hover:   #7b77f5;  
  --accent-muted:   rgba(110, 106, 240, 0.15);  
  
  /* Priority Colors */  
  --priority-urgent: #f04e4e;  
  --priority-high:   #f07a4e;  
  --priority-medium: #f0c44e;  
  --priority-low:    #4ea8f0;  
  --priority-none:   #52525e;  
  
  /* Status / Semantic */  
  --success: #4ecf7a;  
  --warning: #f0c44e;  
  --danger:  #f04e4e;  
  --info:    #4ea8f0;  
  
  /* Shadows */  
  --shadow-sm: 0 1px 3px rgba(0,0,0,0.4);  
  --shadow-md: 0 4px 16px rgba(0,0,0,0.5);  
  --shadow-lg: 0 12px 40px rgba(0,0,0,0.6);  
  --shadow-glow: 0 0 0 3px rgba(110, 106, 240, 0.3);  
}
```

1.2 Typography

Install via Google Fonts or Fontsource:

- **Display / UI:** Geist (clean, modern, made for developer tools — distinct from Inter)
- **Monospace accent:** Geist Mono (task IDs, code snippets, timestamps)

CSS

```
/* tailwind.config.ts */
fontFamily: {
  sans: ['Geist', 'sans-serif'],
  mono: ['Geist Mono', 'monospace'],
}

/* Type scale */
--text-xs: 11px; /* Tags, badges, timestamps */
--text-sm: 13px; /* Sidebar labels, table cells */
--text-base: 14px; /* Body default */
--text-md: 16px; /* Section headings */
--text-lg: 20px; /* Page titles */
--text-xl: 28px; /* Auth headings */
```

1.3 Spacing & Radius

```
Spacing scale: 4px base (Tailwind default works fine)
Sidebar width: 240px
Task card: rounded-lg (8px)
Modals: rounded-xl (12px)
Buttons: rounded-md (6px)
Badges/tags: rounded-full
```

1.4 Motion Tokens

Keep animation purposeful and fast. Users are in a work context.

CSS

```
--duration-fast:    100ms;    /* Hover states */
--duration-normal:  200ms;    /* State transitions */
--duration-slow:    350ms;    /* Modals, page transitions */
--ease-out:         cubic-bezier(0.16, 1, 0.3, 1);    /* Entrances */
--ease-in:          cubic-bezier(0.7, 0, 1, 1);        /* Exits */
--ease-spring:      cubic-bezier(0.34, 1.56, 0.64, 1); /* Drag-drop snap */
```

2. Route Map

```
/                → Landing page (unauthenticated)
/login           → Login
/register        → Register
/onboarding      → Create first workspace (post-register flow)

/app             → App shell (requires auth)
/app/workspaces  → Workspace list / switcher (if no active WS)
/app/:wsSlug     → Workspace home (project list)
/app/:wsSlug/settings → Workspace settings
/app/:wsSlug/members → Member management
/app/:wsSlug/labels → Label management

/app/:wsSlug/:projId → Project kanban board ← primary view
/app/:wsSlug/:projId/list → Project list view (optional enhancement)

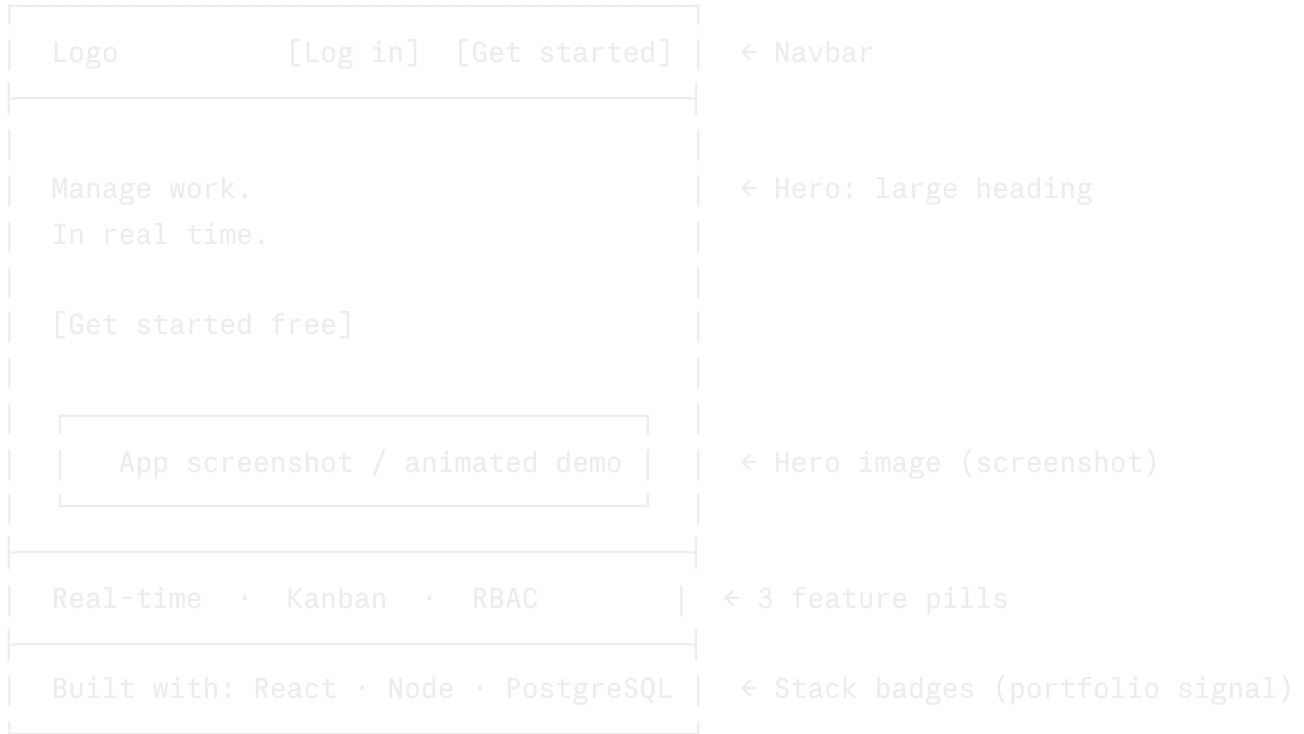
/profile        → User profile & preferences
/404            → Not found
```

3. Pages

3.1 Landing Page (1)

Purpose: Convert visitors. Keep it minimal — this is a portfolio project, not a SaaS product. One strong hero, a short feature list, a CTA.

Layout:



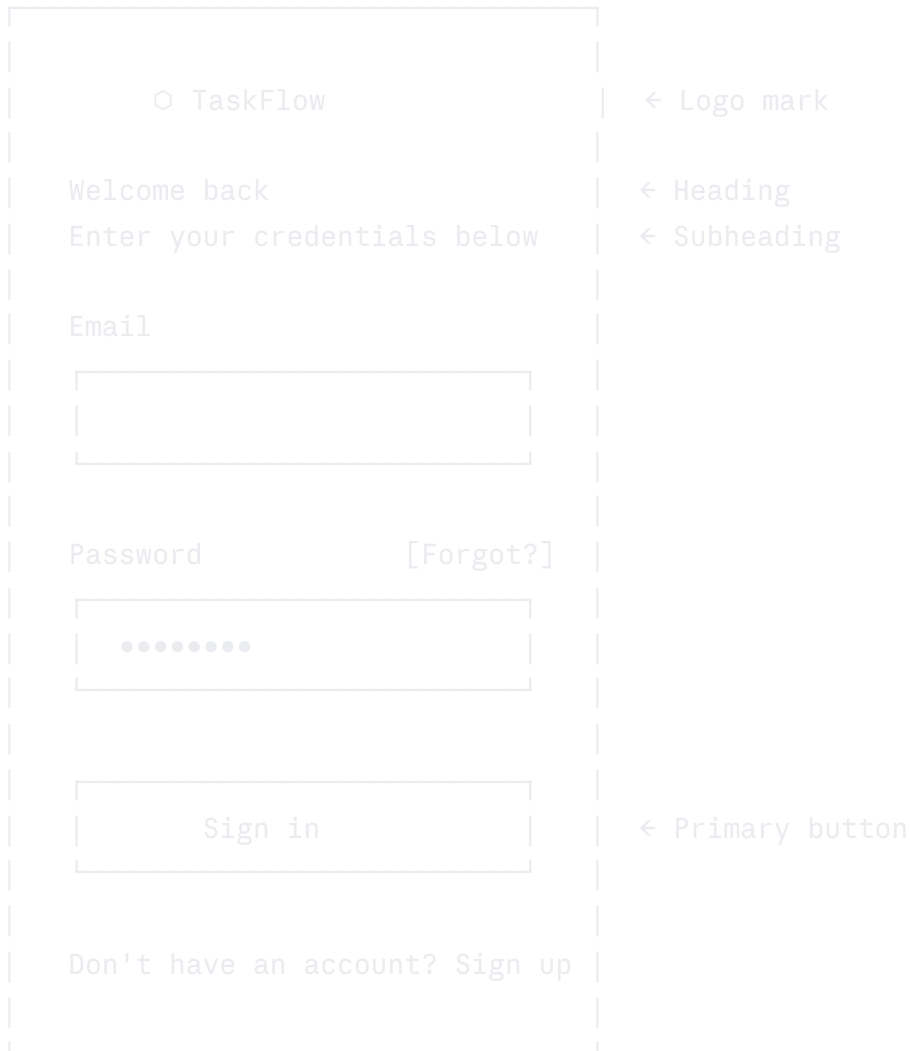
Components: `Navbar`, `HeroSection`, `FeaturePill`, `StackBadge`, `Footer`

Notes:

- Dark background matching the app itself — no jarring color switch on login
- Subtle animated gradient mesh in the hero background
- The screenshot/mockup of the kanban board is important — record it after Phase 3

3.2 Login Page (`/login`)

Layout: Centered card, full-height page with subtle noise texture background.



Components: `AuthCard`, `FormField`, `PasswordField` (with show/hide toggle), `PrimaryButton`, `AuthLink`

Behavior:

- On submit: button shows spinner, disables input
- On error: shake animation on card + inline error message under the field
- On success: brief checkmark flash → redirect

Form validation (client-side, Zod):

```
ts

const loginSchema = z.object({
  email: z.string().email('Enter a valid email'),
  password: z.string().min(1, 'Password is required'),
});
```

3.3 Register Page (/register)

Same card layout as login. Fields: Name, Email, Password, Confirm Password.

Extra behavior:

- Password strength indicator bar (4 segments: weak → strong)
- Inline validation on blur, not on every keystroke
- On success → redirect to /onboarding

Password strength logic:

ts

```
const getStrength = (password: string) => {
  let score = 0;
  if (password.length >= 8)      score++;
  if (/[A-Z]/.test(password))    score++;
  if (/[0-9]/.test(password))    score++;
  if (/^[A-Za-z0-9]/.test(password)) score++;
  return score; // 0-4
};
// Map: 0=none, 1=weak(red), 2=fair(orange), 3=good(yellow), 4=strong(green)
```

3.4 Onboarding Page (/onboarding)

Shown once after registration to create the first workspace. Single focused step — not a multi-step wizard (keep it simple).

Create your workspace
You can always add more later

Workspace name

Acme Corp

Create workspace →

Behavior: On success → navigate to `/app/:wsSlug`

3.5 App Shell (persistent layout for `/app/*`)

The shell wraps every authenticated page. It does not unmount on navigation.

Side
bar

Header

<Outlet /> (page content)

Sidebar content:

○ TaskFlow	← Logo
[Workspace switcher]	← Dropdown: list of user's workspaces
Projects	← Section label
▸ Marketing	
▸ Backend API	← Each links to /app/:wsSlug/:projId
+ New project	
Workspace	← Section label
Members	
Labels	
Settings	
[Avatar] Biswash ▾	← User menu at bottom

Sidebar state (Zustand):

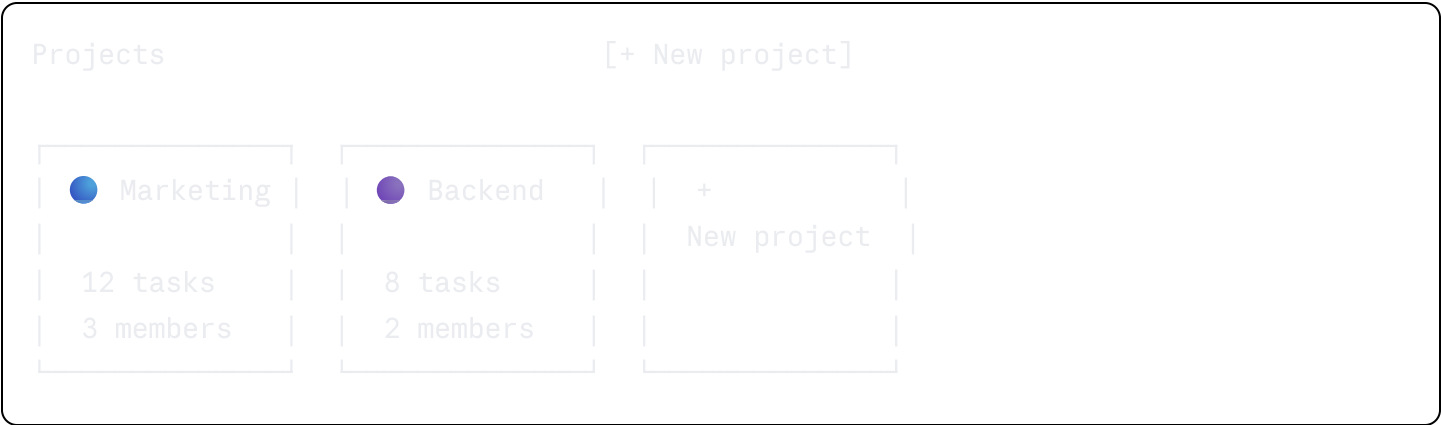
```
ts
// store/ui.store.ts
interface UIState {
  sidebarCollapsed: boolean;
  activeWorkspaceId: string | null;
  activeProjectId: string | null;
  toggleSidebar: () => void;
}
```

Header content:

- Breadcrumb: `Workspace › Project`
- Search icon (opens command palette modal)
- Notification bell with unread badge
- User avatar

3.6 Workspace Home (`/app/:wsSlug`)

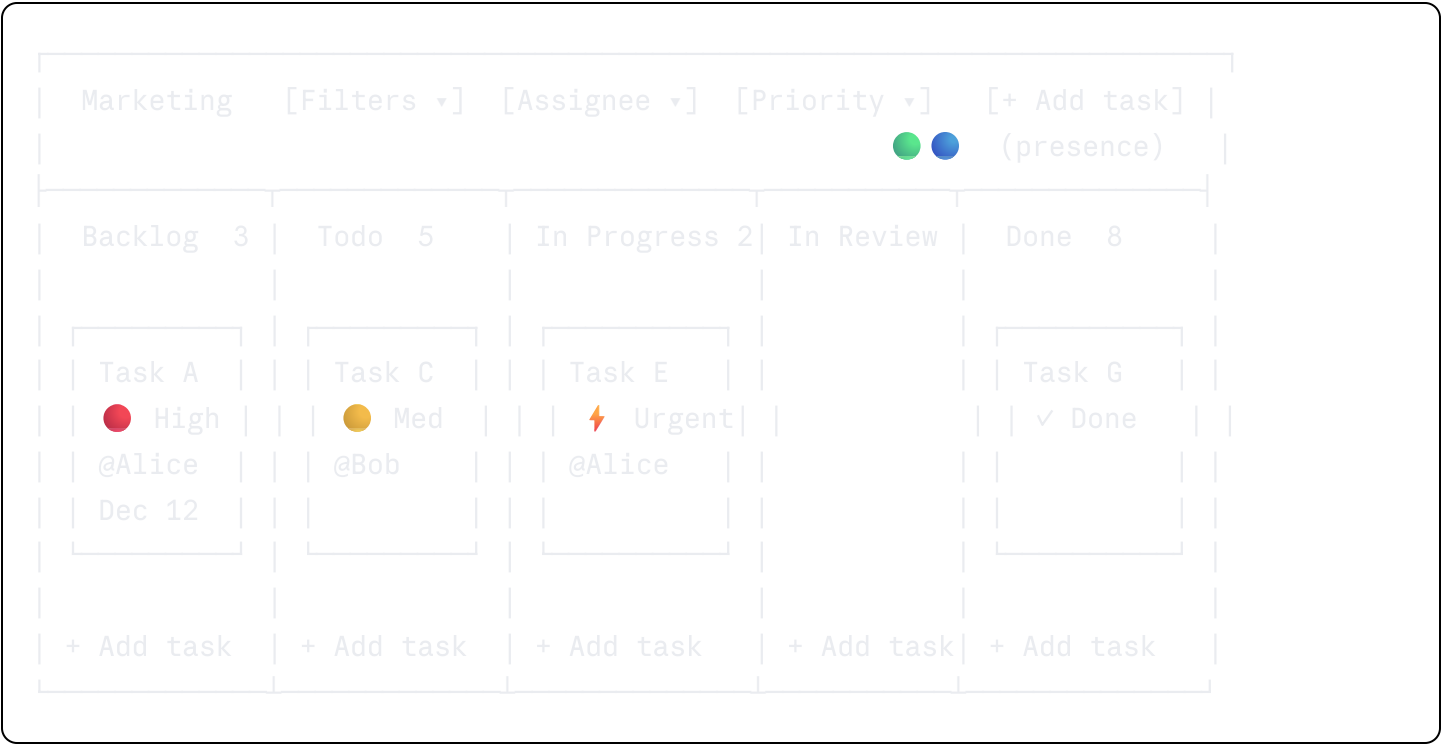
A grid of project cards with a "New project" card at the end.



Project card: Shows name, colour dot, task count. Hover: subtle lift shadow. Click: navigate to board.

3.7 Kanban Board (/app/:wsSlug/:projId) ← Primary View

This is the most important page. Get it right.



Task Card component:



```
interface TaskCardProps {
  task: Task;
  isDragging?: boolean;
}

// Visual elements on the card:
// - Priority colour bar (left edge, 3px wide)
// - Title (2 lines max, truncate)
// - Priority badge (icon + label)
// - Assignee avatar (small, 20px)
// - Due date (red if overdue)
// - Label chips (coloured pill, max 2 visible + "+N more")
// - Comment count icon (if any)
```

Drag behaviour:

- Card lifts with shadow + slight scale(1.02) on grab
- Placeholder ghost shows drop target
- Column highlight on hover-over
- On drop: optimistic update via React Query, PATCH request fires

Filter bar (Zustand — client state):

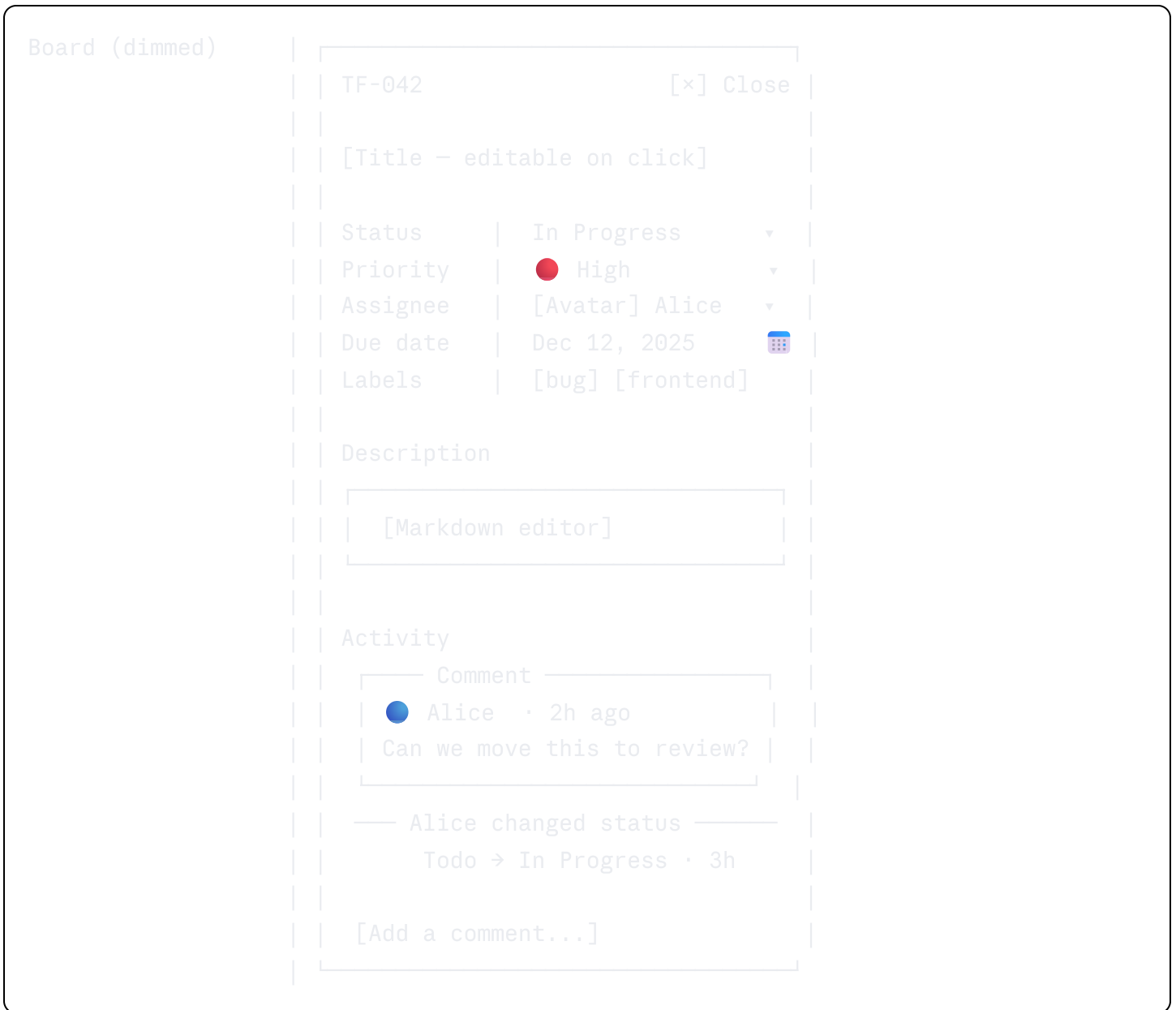
```
ts

// store/board.store.ts
interface BoardFilters {
  assigneeIds: string[];
  priorities: Priority[];
  labelIds: string[];
  dueBefore: string | null;
  dueAfter: string | null;
}
```

Filters are applied client-side on the cached task list — no refetch needed unless the filter set changes.

3.8 Task Detail (Slide-over panel)

Opens from clicking a task card. **Do not navigate to a new route.** Use a right-side slide-over panel so the board stays visible.



Behaviour:

- Opens with slide-in from right (350ms ease-out)
- Clicking outside closes it
- Editable fields: click to edit, blur to save (PATCH on blur)
- Task ID (TF-042) is monospace styled, copyable on click
- Activity log is a chronological feed mixing comments and system events
- System events (status change, assignment) are styled differently from comments

3.9 Workspace Settings (`/app/:wsSlug/settings`)

Simple settings page. Two sections:

General:

- Workspace name (editable)
- Workspace slug (read-only, shown for reference)
- Danger zone: Delete workspace (requires typing workspace name to confirm)

Members tab (`/app/:wsSlug/members`):

Members (4) [Invite member]

[Avatar]	Biswash Khanal	you@email.com	Owner	—
[Avatar]	Alice Smith	alice@...	Member	▼
[Avatar]	Bob Jones	bob@...	Viewer	▼

- Role is a dropdown for Owner only
- Remove button (×) on hover
- Invite opens a modal

3.10 Profile & Preferences (`/profile`)

Profile

Name

[Biswash Khanal]

Email

[you@email.com] (read-only)

Avatar

[BK] (initials, auto-generated)

Security

Current password

[●●●●●●]

New password

[]

Confirm password

[]

[Update password]

Preferences

Theme

(●) Dark () Light

← store in localStorage

4. Component Reference

4.1 Buttons

```
tsx

// Variants:
<Button variant="primary">    // Accent fill, white text
<Button variant="secondary"> // bg-surface, border, text-primary
<Button variant="ghost">      // Transparent, text-secondary, hover:bg-hover
<Button variant="danger">     // Red fill for destructive actions

// Sizes:
<Button size="sm">    // h-7, text-xs, px-3
<Button size="md">    // h-8, text-sm, px-4 ← default
<Button size="lg">    // h-10, text-base, px-5

// States:
<Button loading>      // Spinner replaces content, disabled
<Button disabled>     // Opacity 50%, not-allowed cursor
```

4.2 Form Fields

```
tsx

<FormField
  label="Email"
  error="Enter a valid email" // Shows below input in red
  hint="We'll never share this" // Shows below in muted text
>
  <Input placeholder="you@example.com" />
</FormField>

// Input styles:
// bg-surface, border-default, rounded-md, h-9, px-3
// Focus: border-accent, shadow-glow (the 3px accent ring)
// Error: border-danger, focus shadow in red
```

4.3 Avatar

```
tsx
```

```
<Avatar user={user} size="sm" /> // 24px – used in task cards, comments
<Avatar user={user} size="md" /> // 32px – sidebar user menu, member list
<Avatar user={user} size="lg" /> // 40px – profile page

// Renders: coloured circle with initials (auto-generated from name)
// Color: deterministic from user ID (pick from 8 preset hues)
// Tooltip: user's full name on hover
```

4.4 Badge / Tag

```
tsx

// Priority badge
<PriorityBadge priority="high" />
// Renders: ● with label, or icon-only variant for compact spaces

// Label chip
<LabelChip label={{ name: 'bug', colour: '#f04e4e' }} />
// Renders: pill with coloured left stripe or background tint

// Status badge (for notifications, activity log)
<StatusBadge status="In Progress" />
```

4.5 Empty States

Every list view needs an empty state. Make them useful, not just decorative.

```
tsx

// Pattern:
<EmptyState
  icon={<ClipboardList />}
  title="No tasks yet"
  description="Create your first task to get started."
  action={<Button variant="primary">Add task</Button>}
/>

// Use cases:
// - No projects in workspace
// - No tasks in a column
// - No notifications
// - No search results (different: show the query "No results for "xyz"")
```

4.6 Skeleton Loaders

Never show a spinner for list content. Use skeletons.

tsx

```
// Task card skeleton
<div className="animate-pulse bg-bg-surface rounded-lg h-24 w-full" />

// Use Tailwind's animate-pulse for the shimmer effect
// Show 3 skeletons per column on initial board load
// Remove when React Query's isLoading becomes false
```

5. Toasts

Use `react-hot-toast` or build a lightweight custom solution. Toasts appear in the bottom-right corner, stack upward.

Toast Types

tsx

```
// Success – task moved, comment posted, member invited
toast.success('Task moved to In Progress')

// Error – API failure, validation failure
toast.error('Failed to update task. Try again.')

// Info – real-time event when you're not the actor
toast('Alice moved "Fix login bug" to Done', {
  icon: '➔',
})
```

Toast Design

CSS


```

/* Custom toast styles */
.toast-base {
  background: var(--bg-overlay);
  border: 1px solid var(--border-default);
  color: var(--text-primary);
  border-radius: 8px;
  font-size: 13px;
  padding: 10px 14px;
  box-shadow: var(--shadow-lg);
  max-width: 360px;
}

.toast-success { border-left: 3px solid var(--success); }
.toast-error    { border-left: 3px solid var(--danger); }
.toast-info     { border-left: 3px solid var(--accent); }

```

Toast Rules

- Duration: success = 3s, error = 5s, info = 4s
 - Max 3 visible at once (react-hot-toast handles this)
 - Do NOT toast on every drag-and-drop — only on error or non-obvious actions
 - Do NOT toast on page navigation
-

6. Modals

Use a single `<Modal>` base component. Build specific modals on top of it.

Base Modal Pattern

```

tsx

// Rendered in a Portal to document.body
// Backdrop: bg-black/60, backdrop-blur-sm
// Container: bg-bg-overlay, border-border-default, rounded-xl, shadow-lg
// Animation: fade in backdrop + scale(0.96)→scale(1) on the card (200ms)
// Close: Escape key, clicking backdrop, or explicit close button

<Modal isOpen={isOpen} onClose={onClose} title="Invite member">
  {/* content */}
</Modal>

```

Specific Modals

Invite Member Modal:

Invite to workspace

Email address

[]

Role

☐ Member ☐ Viewer

[Cancel]

[Send invite]

New Project Modal:

New project

Name

[]

Description (optional)

[]

Colour

(colour picker row)

[Cancel]

[Create project]

Delete Confirmation Modal:

Delete "Marketing" workspace?

This will permanently delete the workspace and all its projects and tasks. This cannot be undone.

Type "Marketing" to confirm:

[]

[Cancel]

[Delete workspace] ← danger button

- The delete button stays disabled until the typed text matches exactly

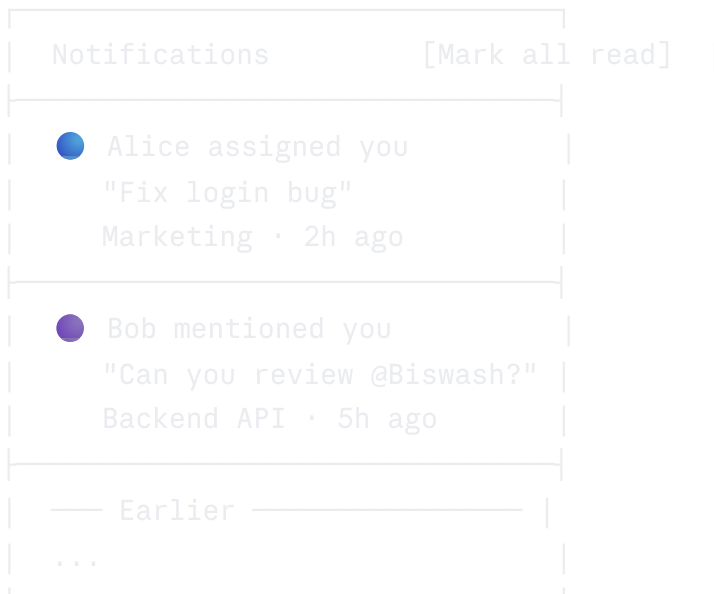
- This pattern is used for: workspace deletion, project deletion

Create Task Inline (in column footer):

- NOT a modal — an inline card that appears below the last task in a column
 - Single input for title, Enter to submit, Escape to cancel
 - Expands to show priority + assignee pickers after typing begins
-

7. Notification Panel

Slides in from the right when the bell icon is clicked. Not a page — an overlay panel.



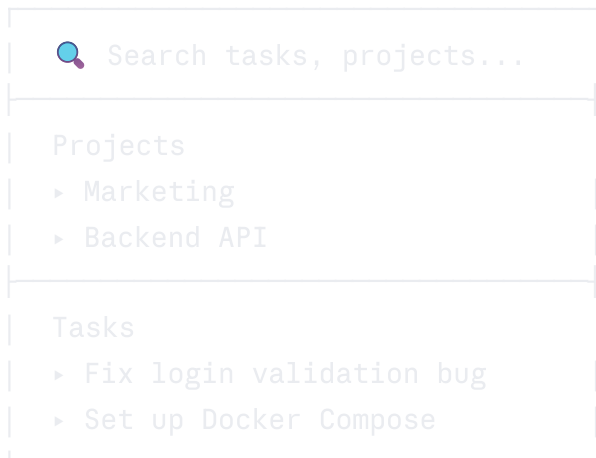
- Unread notifications have a slightly lighter background
 - Clicking a notification → closes panel, navigates to the task → opens task slide-over
 - Bell icon shows a red badge with count (max "9+")
 - New notification received via Socket.io: panel entry slides in at top if open, badge increments if closed
-

8. Command Palette (Nice-to-have, high impact)

Triggered by **Cmd+K**. Dark overlay with centered search input. Filters across:

- Projects

- Tasks (by title)
- Members
- Actions (e.g., "Invite member", "New project")



Use `cmdk` library — it handles keyboard navigation, filtering, and accessibility out of the box.

9. Real-Time UI Patterns

This is where TaskFlow differentiates. When a Socket.io event arrives, update the UI cleanly.

Task moved by someone else

```
tsx

// In your socket event handler:
socket.on('task:moved', ({ taskId, columnId, orderKey }) => {
  queryClient.setQueryData(['tasks', projectId], (old: Task[]) =>
    old.map(t => t.id === taskId ? { ...t, columnId, orderKey } : t)
  );
  // Optional: briefly flash the card border in accent colour
});
```

New task created by someone else

- Card slides in at the bottom of the column (CSS: slide-down animation, 300ms)
- Show a subtle "New" badge on the card for 3 seconds, then fade it out

- Show avatar clusters in the board header for users currently viewing the board
- Animate in/out with scale transition when users join/leave

Connection state banner

```
tsx

// When socket disconnects:
<Banner variant="warning">
  Connection lost. Reconnecting... [Refresh]
</Banner>
// Appears at the top of the board, slides down 40px
// Auto-dismisses when socket reconnects
```

10. Loading & Error States

Route-level loading

Wrap with React Suspense + a skeleton of the target page. Don't use a full-page spinner.

Error Boundary

Create a top-level `<ErrorBoundary>` component that catches rendering errors:

```
tsx

<ErrorFallback
  title="Something went wrong"
  description="We couldn't load this page."
  action={<Button onClick={reset}>Try again</Button>}
/>
```

Async operation states (React Query pattern)

```
tsx
```

```
const { data, isLoading, isError, error } = useQuery(...)

if (isLoading) return <BoardSkeleton />
if (isError) return <ErrorState message={error.message} />
return <KanbanBoard tasks={data} />
```

11. Zustand Store Structure

```
ts

// src/store/ui.store.ts - UI state only
interface UIStore {
  sidebarCollapsed: boolean;
  activeWorkspaceId: string | null;
  activeProjectId: string | null;
  taskDetailId: string | null; // Which task slide-over is open
  notificationPanelOpen: boolean;
  commandPaletteOpen: boolean;

  boardFilters: {
    assigneeIds: string[];
    priorities: Priority[];
    labelIds: string[];
  };

  // Actions
  openTaskDetail: (taskId: string) => void;
  closeTaskDetail: () => void;
  toggleSidebar: () => void;
  setBoardFilter: (key: keyof BoardFilters, value: any) => void;
  clearBoardFilters: () => void;
}
```

12. React Query Key Conventions

Consistent query keys prevent stale cache bugs:

```
ts
```

```
// Query key factory – put in src/lib/queryKeys.ts
export const queryKeys = {
  workspaces: () => ['workspaces'] as const,
  workspace: (wsId: string) => ['workspaces', wsId] as const,
  members: (wsId: string) => ['workspaces', wsId, 'members'] as const,
  projects: (wsId: string) => ['workspaces', wsId, 'projects'] as const,
  project: (projId: string) => ['projects', projId] as const,
  tasks: (projId: string) => ['projects', projId, 'tasks'] as const,
  task: (taskId: string) => ['tasks', taskId] as const,
  notifications: () => ['notifications'] as const,
};
```

13. Accessibility Baseline

These are non-negotiable for a portfolio project — interviewers notice:

- All interactive elements are keyboard-focusable
- Focus ring visible (use the `--shadow-glow` variable, never `outline: none` without a replacement)
- Modal traps focus when open (use `focus-trap-react` or the built-in trap in Radix primitives)
- Drag-and-drop has keyboard alternative (dnd-kit supports this)
- Color is never the sole means of conveying information (priority shows icon + text, not just colour)
- `aria-label` on icon-only buttons (notification bell, close buttons, avatar buttons)
- `role="status"` on toast container

14. Recommended Libraries (Frontend)

Library	Purpose	Notes
<code>react-hot-toast</code>	Toasts	Lightweight, easy to style
<code>@radix-ui/react-*</code>	Dropdowns, dialogs	Headless, accessible primitives
<code>cmdk</code>	Command palette	If implementing Cmd+K

Library	Purpose	Notes
<code>@tiptap/react</code>	Rich text / markdown editor	For task descriptions & comments
<code>date-fns</code>	Date formatting	<code>formatDistanceToNow</code> for "2h ago"
<code>clsx</code> + <code>tailwind-merge</code>	Conditional classnames	<code>cn()</code> utility
<code>focus-trap-react</code>	Modal focus trapping	Or use Radix Dialog which includes this
<code>lexorank</code>	Task ordering	For drag-and-drop order persistence

TaskFlow Frontend Guide — Companion to SRS v1.0